Programação Estruturada em C

Aula 3 — Estruturas de repetição: while, for e do-while

Prof. Me. Eng. Matheus Kotaki

O que muda a partir de hoje?

Programa com decisão

- recebe dados
- avalia uma condição
- escolhe um caminho
- segue adiante

Programa com repetição

- recebe dados
- avalia uma condição
- executa um bloco
- muda o estado e testa novamente

Uma condição também pode controlar uma repetição

Na Aula Anterior:

```
if (idade >= 18) {
    printf("Maior de
idade\n");
}
```

if: devo executar agora?

Agora:

```
while (condicao) {
    // executa
    // muda o estado
}
```

while: devo executar novamente?

Problema: mostrar de 1 até 100

```
printf("1\n");
printf("2\n");
printf("3\n");
```

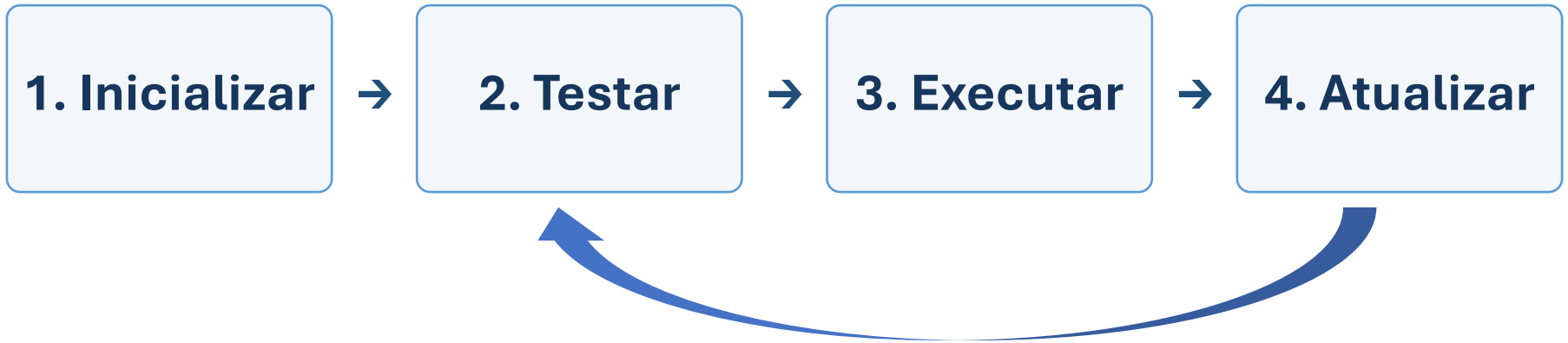
Uma solução literal que funciona para poucos valores, mas repete o mesmo padrão de comando.

O que varia? O número exibido.

O que se repete? Exibir e avançar.

Se fossem 10.000 valores, ainda escreveríamos um printf para cada um?

A anatomia de uma repetição



estado inicial → condição → ação → novo estado → condição novamente

Construindo o primeiro while

```c
int numero = 1;

while (numero <= 5) {
    printf("%d\n", numero);
    numero++;
}
```

Estado inicial	`numero = 1`
Condição	`numero <= 5`
Ação	`printf(...)`
Atualização	`numero++`

Qualquer variável delcarada dentro laço de repetição só existe no context dele

Rastreamento

Iteração	numero atual	numero <= 5	print	numero depois
1	1	verdadeiro	1	2
2	2	verdadeiro	2	3
3	3	verdadeiro	3	4
4	4	verdadeiro	4	5
5	5	verdadeiro	5	6
—	6	falso	—	—

```
int numero = 1;

while (numero <= 5) {
    printf("%d\n", numero);
    numero++;
}
```

A última verificação acontece — mas não produz uma nova iteração.

Quando o estado não muda...

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
}
```

i não se altera

i <= 5 continua verdadeiro

Resultado: laço infinito

Incremento e Decremento

```
int main() {
  int x = 5, y;
  y = ++x;
  printf("x = %d, y = %d", x,y);
}
```

Incrementa x e depois salva em y

x = 6, y = 6

```
int main() {
  int x = 5, y;
  y = x++;
  printf("x = %d, y = %d", x,y);
}
```

Incrementa x e depois salva em y

x = 6, y = 5

Atribuição Composta

Operador	Nome	Exemplo	Equivalente a
+=	Atribuição de adição	`x += y;`	`x = x + y;`
-=	Atribuição de subtração	`x -= y;`	`x = x - y;`
*=	Atribuição de multiplicação	`x *= y;`	`x = x * y;`
/=	Atribuição de divisão	`x /= y;`	`x = x / y;`
%=	Atribuição de módulo (resto)	`x %= y;`	`x = x % y;`

Atualizar não basta

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
    i--;
}
```

Há atualização, mas ela move i no sentido oposto ao encerramento.

1 → 0 → -1 → -2 → ...

A condição continua verdadeira.

Limites: < ou <= muda o comportamento

i < 5

```
int i = 1;

while (i < 5) {
    printf("%d ", i);
    i++;
}
```

Saída: 1 2 3 4

i <= 5

```
int i = 1;

while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

Saída: 1 2 3 4 5

Contador e acumulador têm papéis diferentes

```c
int soma = 0;
int i = 1;

while (i <= 5) {
    soma = soma + i;
    // soma += i;
    i++;
}

printf("%d\n", soma);
```

`i`	controla a progressão
`soma`	preserva o total acumulado
`soma = 0`	define o estado inicial da soma

Rastreando uma acumulação

Iteração	i	soma atual	Cálculo	soma depois
1	1	0	0 + 1	1
2	2	1	1 + 2	3
3	3	3	3 + 3	6
4	4	6	6 + 4	10

```
while (i <= 5) {
    soma = soma + i;
    // soma += i;
    i++;
}
```

Nem toda repetição é uma contagem

```
char resp = 's';

while (resp == 's' || resp == 'S') {
    printf("Preso no Loop");
    printf("Deseja permanecer no Loop?");
    scanf(" %c", &resp);
}
printf("SAIU!");
```

Atualização do estado

nova leitura

Condição de parada

`resp != 's' ou 'S'`

Prática 1

```
int i = 2;

while (i <= 10) {
    printf("%d ", i);
    i += 2;
}
```

- Preveja a saída antes de executar.
- Rastreie os valores de i.
- Modifique para contar de 10 até 2.

for

while / for: o mesmo mecanismo

while

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
    i++;
}
```

for

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

inicialização • condição • atualização

As três partes do for

```
for (int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

`int i = 1`

INICIALIZAÇÃO

`i <= 10`

CONDIÇÃO

`i++`

ATUALIZAÇÃO

A variável i só existe no contexto do for

Qualquer variável delcarada dentro laço de repetição só existe no context dele

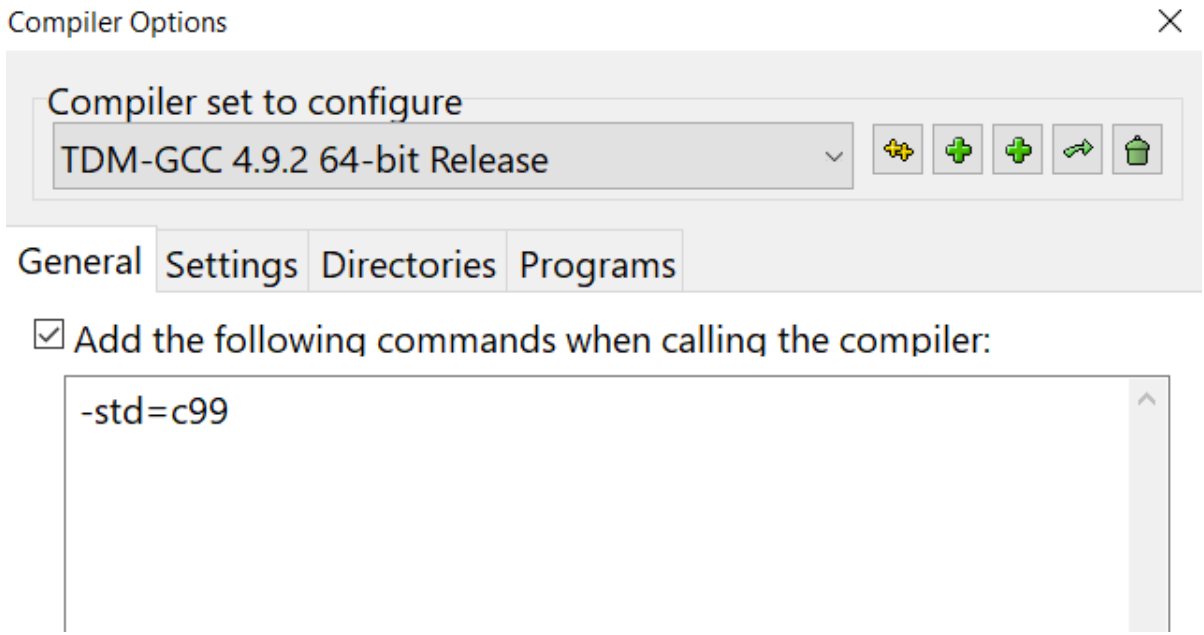
Dev-C++: habilite C99 ou posterior

Nesta disciplina, vamos declarar a variável diretamente no for:

```
for (int i = 0; i < 10; i++) {
    // ...
}
```

Configuração necessária

- usar C99 ou padrão posterior



O for pode expressar diferentes progressões

```
for (int i = 1; i <= 5; i++)          →    1, 2, 3, 4, 5
```

```
for (int i = 10; i >= 2; i -= 2)      →    10, 8, 6, 4, 2
```

```
for (int i = 0; i < 10; i += 3)       →    0, 3, 6, 9
```

O for pode expressar diferentes progressões

```
for(int a = 0, b = 0; a+b<50; a++, b++){
      printf("%3d\n", a+b);
}
```

> %3d
>
> Reserva 3 espaços para o número inteiro

```
  0
  2
  4
  6
  8
 10
 12
 14
 16
 18
 20
 22
 24
 26
 28
 30
 32
 34
 36
 38
 40
 42
 44
 46
 48
```

do-while

Um caso natural para do-while

```
int opcao;

do {
    printf("1 - Cadastrar\n");
    printf("2 - Consultar\n");
    printf("0 - Sair\n");
    scanf("%d", &opcao);
} while (opcao != 0);
```

primeiro executa

↓

depois testa

Consequência:

o corpo executa pelo menos uma vez

Atenção: aqui o ; após while(condicao) faz parte da sintaxe.

Escolha funcional: não há uma regra rígida

`while`

Condição testada antes.
Boa leitura quando a repetição depende de um estado que muda.

`do-while`

Bloco executa antes do primeiro teste.
Útil quando a primeira execução é necessária.

`for`

Reúne inicialização, condição e atualização.
Frequentemente claro em progressões.

Mais de uma estrutura pode resolver corretamente o mesmo problema.

Um ponto e vírgula pode mudar tudo

for com comando vazio

```
for (int i = 1; i <= 5; i++);
{
    printf("Teste\n");
}
```

O ; encerra o comando de repetição.

while com comando vazio

```
int i = 1;
while (i <= 5);
{
    printf("%d\n", i);
    i++;
}
```

Pode surgir um laço vazio infinito.

Prática 2

Desenvolva um programa que leia 5 números inteiros e informe:

- a soma de todos os valores;
- quantos valores são positivos;
- quantos valores são pares.

Possível esqueleto da solução

```
int numero;
int soma = 0;
int positivos = 0;
int pares = 0;

for (int i = 1; i <= 5; i++) {
  //leitura
  //soma
  //verifica positivo
  //verifica par
}

printf(...
```

Exercícios Complementares

```
for (int i = 2; i <= 8; i +=
2) {
    printf("%d ", i);
}
```

Qual é a saída?

```
int x = 5;
while (x > 0) {
    printf("%d ", x);
    x++;
}
```

Esse laço termina? Por quê?

Ao ler qualquer laço simples, pergunte

1. Qual é o estado inicial?

2. Qual condição permite continuar?

3. O que acontece em cada repetição?

4. O que muda para que ele termine?